

A3 (16%) Submission due on 17 April 2026 (Week 13 Friday)

Purpose of Assignment 3

Gradient descent is one of the fundamental technologies behind modern AI, powering the training of most modern machine learning models, especially deep learning models such as neural networks. Implementing gradient descent correctly once is not enough. A practitioner must understand what happens when the algorithm is given a bad learning rate, a poor starting point, or a tricky loss landscape. In this assignment, you will run gradient descent under controlled conditions and return numerical answers that demonstrate your understanding of its behaviour.

Function Interface

Your file must expose one function with the following signature:

```
def A3_<MatricNumber>(task: str, params: dict) -> dict:
    """
    Parameters
    -----
    task      : one of "A", "B", "C"
    params    : dict of task-specific inputs (see each task)
    Returns
    -----
    dict of task-specific outputs - keys must match exactly
    """
```

The grader will call your function once per task, check every key in the returned dict, and award marks automatically. Use the output key names exactly as written.

Submission Rules

- Name your file A3_<MatricNumber>.py (e.g., A3_A1234567R.py).
- The function name must also use your matric number: A3_A1234567R.
- Do NOT zip, compress, or submit a notebook.
- Do NOT use `scipy.optimize` or any external optimization library. NumPy only.
- All output values must be returned inside the dict, do not print them.
- Deductions apply for missing keys, wrong types, or wrong array lengths.

Task A — Effect of Learning Rate on Convergence [5 marks]

Concept: Too-large α causes divergence; too-small α converges unnecessarily slowly.

Cost function and setup

$$f(x) = (x - 3)^2 + 5$$

Initialise at $x_0 = 10.0$. The grader will always call with `lr_list = [0.001, 0.1, 0.9, 1.2]` and `num_iters = 200`.

Input params

```
lr_list    : list of floats
num_iters  : int
```

Required output keys

Output key	Type	Description
<code>trajectories</code>	<code>dict</code>	<code>lr</code> → numpy array length <code>num_iters</code> (x value after each step, e.g., the first x is the x value after the 1st iteration, Do not include the initial value x_0)
<code>final_f</code>	<code>dict</code>	<code>lr</code> → float (f evaluated at the last x)
<code>diverging_lrs</code>	<code>list[float]</code>	sorted list of lr values where $ x_{\text{final}} > 1e6$
<code>converged_lrs</code>	<code>list[float]</code>	sorted list of lr values where $f_{\text{final}} < 5.1$
<code>f_final_slow</code>	<code>float</code>	f_{final} for <code>lr = 0.001</code> , rounded to 2 decimal places

Implementation notes

- `diverging_lrs` / `converged_lrs`: return plain Python lists, not numpy arrays.
- `f_final_slow`: apply `round(value, 2)` before returning.
- If a trajectory diverges, `f_final` will be very large but still a valid float — handle normally.

Hint: If you're not familiar with Python dictionaries, you can refer to these resources:

- https://www.w3schools.com/python/python_dictionaries.asp
- <https://docs.python.org/3/tutorial/datastructures.html>

They provide helpful explanations and examples to get you started.

Task B — Local Minima [6 marks]

Concept: Gradient descent follows the local gradient and can get trapped in a local minimum, never reaching the global one.

Cost function and setup

$$f(x) = x^4 - 8x^2 + x + 10$$

The grader will always call with `init_list = [-3.0, -1.0, 0.5, 2.5]`, `lr = 0.01`, `num_iters = 500`.

Input params

```
init_list : list of floats
lr       : float
num_iters : int
```

Required output keys

Output key	Type	Description
final_x	dict	init → float (final x value)
final_f	dict	init → float (f at final x)
trajectories	dict	init → numpy array of x values, length num_iters
left_basin_inits	list[float]	sorted list of init values where x_final < 0
global_min_side	str	"left" or "right": basin with the lower mean f_final
best_f	float	lowest f_final across all inits, rounded to 2 decimal places.

Implementation notes

- `left_basin_inits`: an init belongs to the left basin if its final x is negative. Sort ascending.
- `global_min_side`: compute the mean `f_final` of left-basin runs and of right-basin runs; return "left" or "right" accordingly.
- `best_f`: use `round(value, 2)`.

Note: A basin refers to the region of initial points that all converge to the same local minimum under gradient descent. So while a local minimum is a point, a basin is the area around it that leads to that minimum.

Task C — Sensitivity to Initialisation in 2D [5 marks]

Concept: In higher dimensions different starting points fall into different basins of attraction, yielding different final solutions.

Cost function and setup

$$f(u,v) = \sin(u) \cdot \cos(v) + 0.1(u^2 + v^2)$$

The grader will always call with `init_pairs = [[0.1, 0.1], [2.0, 2.0], [-1.5, 3.0], [3.5, -2.0]]`, `lr = 0.05`, `num_iters = 500`.

Input params

```
init_pairs : list of [u0, v0] pairs
lr         : float
num_iters  : int
```

Required output keys

Output key	Type	Description
final_uv	dict	<code>str(init_pair) → [final_u, final_v]</code>
final_f	dict	<code>str(init_pair) → float</code>
num_basins	int	number of distinct basins (group by <code>round(f_final, 1)</code>)
same_basin_as_first	list	sorted str keys (excl. "[0.1, 0.1]") that converge to the same basin as "[0.1, 0.1]"
best_init	str	str key of init_pair with lowest final_f

Implementation notes

- Keys must be `str([u0, v0])`, e.g. `"[0.1, 0.1]"`.
- `num_basins`: round each `final_f` to 1 decimal place, count unique values.
- `same_basin_as_first`: the first `init_pair` is `[0.1, 0.1]`. Among the remaining three `init_pairs`, return a list containing the str key(s) (excluding `'[0.1, 0.1]'`) whose `final_f` value, when rounded to 1 decimal place, is the same as that of `'[0.1, 0.1]'`.
- `best_init`: return the string key with minimum `final_f`.

Mark Breakdown

Part	Description	Marks
A	trajectories (1 mk) · final_f (1 mk) · diverging_lrs (1 mk) · converged_lrs (1 mk) · f_final_slow (1 mk)	5
B	final_x (1 mk) · final_f (1 mk) · trajectories (1 mk) · left_basin_inits (1 mk) · global_min_side (1 mk) · best_f (1 mk)	6
C	final_uv (1 mk) · final_f (1 mk) · num_basins (1 mk) · same_basin_as_first (1 mk) · best_init (1 mk)	5
	Total	16

All outputs are checked automatically. Marks are awarded per output key. A correct gradient descent implementation with a wrong key name scores zero for that key.

How the Grader Will Call Your Code

```
import A3_A1234567R as student

# Task A
out = student.A3_A1234567R('A', {'lr_list': [0.001,0.1,0.9,1.2],
'num_iters': 200})
# checks: out['diverging_lrs'], out['converged_lrs'],
out['f_final_slow'] ...

# Task B
out = student.A3_A1234567R('B', {'init_list':[-3.,-
1.,0.5,2.5],'lr':0.01,'num_iters':500})
# checks: out['left_basin_inits'], out['global_min_side'],
out['best_f'] ...

# Task C
out = student.A3_A1234567R('C', {'init_pairs':[[0.1,0.1],[2.0,2.0],[-
1.5,3.0],[3.5,-2.0]],'lr':0.05,'num_iters':500})
```

Please remember to download and double check if you submit the correct file.

Submission folder: Canvas/EE2211/Assignments/A3

The submission deadline is 23:59, 17 April 2026 (Friday).

No extensions will be granted.

Late submission policy:

Up to 12 hours late: 80% of your grade

1 day late: 50% of your grade

2 days late: 30% of your grade

More than 2 days late: No marks (0%)